

개발 기획 대화록

DESCO 교육 운영 보조 웹앱 시스템 개발 기획

레고 컨베이어 벨트 조립 게임의 기록 측정·운영 보조 시스템
3-레이어 웹앱 (관리자 / FT / 팀 사용자) 설계 대화록

작성일	2026년 5월 15일
참여자	요청자 + Claude (Anthropic)
문서 종류	요구사항 정의 및 설계 검토 대화록
주요 산출물	CLAUDE.md, docs/SPEC.md (Claude Code CLI용)

목차

1. 프로젝트 초기 요구사항
2. 설계 시작 전 검토 — 사전 확인 사항 6개 영역
3. 요구사항 명확화 — 답변 1차
4. 기술 스택 + 시스템 아키텍처 + 데이터 모델 1차
5. 세부 확정 + 신규 요구사항 (관리자 동기화 시작)
6. 데이터 모델 최종 + 화면 구성 + 다음 단계 선택
7. 추가 조정 (TM-03 시작 버튼, 리더보드 팝업 방식)
8. TM-03 측정 화면 + ADM-03 라이브 운영 상세 설계
9. Claude Code CLI 연동 산출물

1. 프로젝트 초기 요구사항

요청자

[필요한 기능]

<관리자>

1. 차수 생성
2. 팀 생성
3. 팀별 로그인 아이디/패스워드
4. 실전 플레이 시간 설정(Default 15분)
5. 각 팀별 기록 테이블(시간, 조립 순서 및 턴수)

<사용자>

1. 타임워치(시작, 정지 버튼)
2. 시도 회차별 기록(텍스트 입력)
3. 조립 순서 및 턴수 입력 기능(조립 순서는 5칸으로 구성 텍스트 입력, 턴수는 $nT + n$ 형태로 n 부분에 숫자 입력)

<FT>

1. 팀 별 미션 성공 여부 체크. 성공 처리된 기록을 공식 기록으로 치며, 관리자 페이지에서 성공한 것은 별도 표시(중간에 멈추고 다시 시도한 로우 데이터도 모두 DB에 쌓임)
2. 기타 팀별 기획 방식 및 기록 사항 기록(랩업 참조용)

위의 3개의 레이어를 가지고 있는 웹앱 시스템을 개발하려고해. 이 시스템은 온라인-오프라인 믹스로 진행되는 교육 콘텐츠 운영을 위한 보조 장치로 사용되는 시스템이야. 관리자는 최상 위 단의 운영 관리자를 의미하고, 사용자는 교육생, FT는 교육 현장에서 운영을 담당하는 인원을 의미해.

시스템 개발을 위한 개발 기획서를 만들어줘. 한번에 결과물을 내지 말고 개발 기획에 있어 필요한 부분을 먼저 검토하고 진행하자.

2. 설계 시작 전 검토 — 사전 확인 사항

요청 내용을 보면 교육 콘텐츠(아마도 팀 단위로 진행하는 조립/퍼즐형 미션)의 운영을 보조하는 3-레이어 웹앱이고, 기록 측정과 사후 분석이 핵심 가치로 보인다. 다만 기획서를 바로 쓰기 전에 짚어야 할 부분이 꽤 있어서, 우선 확인이 필요한 항목들을 6개 영역으로 정리했다.

2.1 교육 콘텐츠의 실체와 운영 맥락

- 실제 교육 미션이 무엇인지 (조립 순서 5칸 + $nT+n$ 표기법을 보면 큐브, 하노이탑, 혹은 특정 조립 키트 같은 것이 떠오르는데, 어떤 활동인가?)
- " $nT+n$ "이 정확히 어떤 의미인지 (예: $3T+2$ = 시계방향 3턴 후 추가 2턴? 큐브 표기처럼 회전 단위인지, 단순 카운트인지)
- 1회 운영 시 동시 참여 팀 수와 차수 운영 빈도
- "온라인-오프라인 믹스"의 정확한 의미

2.2 사용 환경 (디바이스/네트워크)

- 사용자가 사용하는 단말기 — 모바일/태블릿/노트북 중 무엇이 주력인지 (스톱워치 UI 설계가 완전히 달라짐)
- 교육 현장의 네트워크가 안정적인지, 일시 끊김 시 로컬에 임시 저장하고 복구해야 하는지
- 한 팀이 1개 디바이스로 입력하는지, 팀원 각자가 자기 디바이스로 보는지

2.3 인증 및 권한 구조

- 팀 로그인 ID/PW가 팀 단위 공유 계정인지, 아니면 팀 내 개별 사용자인지
- 관리자/FT는 어떤 방식으로 로그인하는지 (관리자 1인 + FT 다수? FT도 차수별로 다른 인원?)
- FT가 팀 생성/시간 변경 같은 권한도 갖는지, 아니면 순수하게 미션 성공 체크와 기록 작성만 하는지

2.4 핵심 게임 규칙 / 기록 규칙

- 15분 디폴트 시간은 1회 시도 기준인지, 전체 세션 누적인지
- 한 팀이 시도할 수 있는 횟수에 제한이 있는지
- 스톱워치는 클라이언트(브라우저) 기준인지, 서버 시간 기준인지
- 미션 성공 판정 기준 — FT의 수동 판정만으로 충분한지, 자동 검증도 필요한지
- 중간에 멈췄다가 다시 시도한 로우 데이터의 "시도 회차" 카운팅 규칙

2.5 데이터 및 사후 활용

- 기록 데이터를 엑셀/CSV로 내보내야 하는지
- 차수 종료 후 데이터 보관 기간
- 팀 간/차수 간 비교 리포트 필요 여부
- 관리자가 실시간으로 진행 상황을 보고 싶은지

2.6 기술 스택 및 배포

- 선호하는 기술 스택이 있는지
- 호스팅 환경 — 회사 내부 인프라인지, Vercel/Cloudflare 같은 외부 클라우드인지
- 내부 직원용 시스템인지, 외부 운영자에게도 배포할 가능성이 있는지
- 개발 일정 — 언제까지 1차 버전이 필요한지

검토 결과

이 중에서 특히 1번(미션의 실체와 nT+n 의미), 3번(팀 로그인 단위), 4번(시간/시도/성공 판정 규칙), 6번(기술 스택과 일정)이 기획서 구조 자체를 바꾸는 결정 사항이라 먼저 정리되어야 한다. 2번과 5번은 가정해서 진행해도 후속 조정 가능하다.

3. 요구사항 명확화 — 답변 1차

요청자 답변

1. 실제 교육 미션: 한 팀은 다섯명으로 구성되고 빨, 노, 초, 파, 흰 색의 다섯가지 색상 3~4종류 총 30개의 레고 블럭을 받게 되고, 한 사람 당 한 가지 색의 레고 블럭을 담당하고 컨베이어 벨트 시스템으로 특정 순서에 맞춰 블럭을 조립하며 제시한 최종 조립품의 색상과 위치까지 동일하게 구성될 수 있도록 기획하고 어떤 팀이 가장 짧은 시간내에 완벽한 완성품을 10개 만드느지를 경쟁하는 게임 콘텐츠.

미션을 성공하는 방법이 10000가지가 넘어 팀별로 레고 색상 조립 순서와 몇 바퀴를 돌아 하나가 완성되는지의 기획이 거의 다 팀별로 다르게 나옴. $nT+n$ 은 예시 의미가 맞음. 어떤 팀이 빨-노-초-파-흰의 순서로 컨베이어 벨트 순서를 기획했고, 빨간색에서 흰색까지 3번 반복 후 노란색에서 완성품 하나가 나온다면 이 팀은 $3T+2$ 로 기록.

1회 운영 시 동시 참여 팀 수는 제한 없음. 차수 운영 빈도는 니즈에 따라 변동이라 특정 어려움. 대부분 교육생은 오프라인에서 활동하고 본 서비스만 사실상 온라인에서 기록 측정용으로 사용.

2. 사용 환경: 모바일 사용 기준. 네트워크 문제 대비 로컬 임시 저장 필요. 한 팀이 1개의 디바이스 사용.

3. 인증: 팀 로그인 아이디 패스워드는 팀에 한 명만 로그인 하므로 팀 내 개별 사용자. 관리자 1인 + FT 다수로 로그인. FT는 팀 생성/시간 변경 권한 없음.

4. 게임 규칙: 15분 디폴트는 전체 세션 누적, 즉 기록 측정 제한시간. 15분 내에는 몇 회든 시도 가능하지만 15분 지나면 전체 게임 종료. 시도 횟수 제한 없음. 스톱워치는 클라이언트 기준. 여러번 시도하는 가운데 교육생 혹은 FT가 직접 해당 팀의 모바일에서 스타트와 스탑 버튼을 누르는 형태. 각 팀마다 매 회수 시작 시점 다를 수 있음. 핸드폰에 표기되는 타이머의 숫자가 기준. 미션 성공 기준은 수동 — 자동 검증 어려움. 스타트와 스탑 버튼 누른 것을 하나의 시퀀스로 카운팅. 그 시퀀스 안에서 FT가 성공 판단 시 그 기록만 공식 기록.

5. 데이터: 엑셀 내보내기 기능 필요. 데이터 1년 보관. 팀 간 비교 리포트 있으면 좋음. 관리자가 실시간으로 진행 상황 확인 필요. 시상을 위해 각 팀 별 최단 기록이 실시간으로 표시되는 테이블을 별도 브라우저로 띄울 수 있으면 좋음.

6. 기술 스택: 추천대로 진행. Cloudflare로 호스팅. 내부 교육 운영용 시스템. 최대한 빠르게 구축 희망.

4. 기술 스택 + 시스템 아키텍처 + 데이터 모델 (1차)

답변 잘 받았다. 시스템 성격이 꽤 명확해졌다. 컨베이어 벨트 레고 게임에서 가장 중요한 것은 "FT가 성공 판정한 시
퀀스만 공식 기록" 이라는 부분과 모바일 1팀 1단말 오프라인 대응, 그리고 시상용 별도 리더보드 세 가지로 보인다.

4.1 기술 스택 추천

Cloudflare 호스팅 + 최단 구축 우선 + 모바일 우선이라는 조건에 맞춰 다음 조합을 추천한다.

프론트엔드

- React + Vite + TypeScript (Next.js보다 단순, Cloudflare Pages에 즉시 배포)
- Tailwind CSS + shadcn/ui — 모바일 UI 컴포넌트 빠르게 조립
- Dexie.js — IndexedDB 래퍼, 오프라인 임시 저장 큐 관리
- TanStack Query — 서버 데이터 캐싱과 폴링 처리

백엔드 / 인프라

- Hono on Cloudflare Workers — Express와 거의 동일한 사용감, Workers 최적화
- Cloudflare D1 (SQLite 호환) — 트랜잭션 지원, 무료 한도 충분
- Drizzle ORM — D1과 궁합 좋고 가벼움
- JWT 기반 인증 — 외부 인증 서비스 불필요

실시간 처리

- 1~2초 폴링 방식으로 시작 권장. 10~20팀 규모에서는 WebSocket보다 안정적
- 시상용 리더보드 화면은 SSE로 처리하면 더 깔끔하지만, MVP는 폴링으로 충분

기타

- ExcelJS — 엑셀 내보내기
- PWA 매니페스트 추가 — 모바일 홈화면 추가 시 앱처럼 동작
- bcrypt 대신 Web Crypto API 기반 PBKDF2 (Workers 환경 호환)

4.2 시스템 아키텍처 개요

```
[팀 모바일] · [FT 모바일/태블릿] · [관리자 PC] · [리더보드 디스플레이]
      ↓ HTTPS
[Cloudflare Pages - 정적 SPA]
[Cloudflare Workers - Hono API]
      ↓
[Cloudflare D1 - 모든 영구 데이터]
```

4종류 UI:

1. 팀 사용자 화면 — 로그인 → 스톱워치 → 시도 기록 입력 → 저장

2. FT 화면 — 담당 팀 시도 목록 확인 → 성공 판정 → 기획/관찰 메모
3. 관리자 화면 — 차수/팀/계정 관리, 시간 설정, 실시간 모니터링, 엑셀 내보내기, 비교 리포트
4. 공개 리더보드 화면 — 차수 코드만으로 진입, 실시간 팀별 최단 기록 테이블

4.3 데이터 모델 (1차)

핵심 6개 테이블로 정리.

users (관리자 + FT 계정)

- id, username (UNIQUE), password_hash, role(admin/ft), display_name, created_at

rounds (차수)

- id, name, time_limit_seconds (기본 900), status(preparing/active/ended), public_code(리더보드 진입 용), created_by, created_at, ended_at

teams (팀 = 1개의 모바일 로그인 단위)

- id, round_id, name, username, password_hash
- session_started_at, session_status
- assigned_ft_id (FK → users, NULL 허용)

attempts (스타트~스탑 1회 = 1행)

- id, team_id, attempt_no
- started_at, stopped_at, duration_ms (클라이언트 측정값이 공식 기록)
- assembly_order_1 ~ assembly_order_5
- turn_t, turn_extra (nT+n의 두 숫자를 분리 저장)
- is_success, success_marked_by, success_marked_at
- user_notes
- client_uuid (오프라인 동기화용 먹통 키)
- created_at

ft_notes (FT 기획/관찰 메모)

- id, team_id, author_id, content, created_at, updated_at

audit_logs (선택, 관리자 행위 추적)

핵심 인덱스

- `attempts(team_id, is_success, duration_ms)` — 리더보드 조회
- `attempts(team_id, attempt_no)` — 시도 이력 조회
- `teams(round_id)` — 차수별 팀 목록
- `teams(username)` — 로그인 조회

4.4 주요 데이터 흐름

- (a) 팀 시도 기록 — 사용자 "시작" → 클라이언트 시각 + client_uuid 발급, IndexedDB에 임시 저장 → "정지" → 조립 순서/턴수/메모 입력 → 서버 POST. 네트워크 실패 시 IndexedDB 큐에 남기고 복귀 시 자동 재시도. client_uuid로 서버에서 역등 처리.
- (b) 15분 카운트다운 — "세션 시작" 시각을 서버에 기록 → 모든 클라이언트가 그 시각 기준으로 카운트다운. 페이지 새로고침이나 단말 교체에도 정확. 종료 후 attempt 저장 시도는 서버에서 거부.
- (c) FT 성공 판정 — FT가 담당 팀의 시도 목록 조회 → 특정 시퀀스에 성공 마킹 → `is_success = true` 갱신. 이 순간 리더보드 캐시 무효화.
- (d) 리더보드 실시간 표시 — 공개 화면에서 public_code로 입장 → 1초 폴링으로 팀별 최단 success 기록 갱신.
- (e) 관리자 실시간 모니터링 — 차수 상세 화면에서 팀별 진행 상태를 2초 폴링으로 표시.

추가 확인 사항

1. 조립 순서 5칸 외에 "최종 완성품의 색상/위치" 정보를 시스템에 따로 입력받아야 하는지
2. 차수 종료 후 팀 계정 처리 — 자동 비활성화 + username 재사용 허용 vs 영구 보존 + round 내 UNIQUE
3. FT-팀 배정 방식 — 모든 팀 보기 가능 vs 명시적 배정
4. 10개 완성 카운트 — 시퀀스별 완성 개수 추적 필요 여부

5. 세부 확정 + 신규 요구사항

요청자 답변

1. 조립 순서도 시스템으로 받아 정리. 팀 별 기록 데이터 마지막 열에 조립 순서 열과 턴 수 열을 만들어 데이터를 받아 자동 기록.
2. (2) — 영구 보존 + round 내 UNIQUE.
3. 모든 팀을 다 볼 수 있도록 하되 소팅을 통해 특정 팀만 디스플레이되도록 하는 기능 만들기.
4. 성공은 10개 다 완성. 이 부분은 오프라인으로 완료되면 FT가 육안으로 확인하고 FT 시스템에서 성공 처리.

신규 요청: 관리자는 15분 게임 시간이 시작되는 스타트 버튼을 관리자 페이지에서 클릭하면 모든 팀의 첫 번째 기록 측정을 위한 스타트 버튼은 동기화되서 같이 눌러져야함. (이 첫 한 번은 교육생이 아닌 관리자가 15분 카운트 다운 시작과 함께 각 팀 기록 측정 스타트가 함께 됨, 그 이후 부터는 팀별 개별 동작)

6. 데이터 모델 최종 + 화면 구성

4가지 결정사항 모두 반영하고, 관리자 동기화 시작은 시스템 동작 방식에 영향을 주는 신규 요구사항이라 별도로 정리한다.

6.1 확정 사항 정리

- 조립 순서 5칸 + 턴수는 attempts 테이블 마지막 컬럼군에 그대로 자동 기록
- 차수 종료 후 팀 데이터 영구 보존, username UNIQUE는 차수 내에서만 (스키마: `teams(round_id, username)` 복합 UNIQUE)
- FT는 전체 팀 조회 가능 + 정렬/필터로 특정 팀 디스플레이 → `teams.assigned_ft_id` 컬럼 삭제
- 성공 = FT 육안 확인 후 마킹, 시퀀스별 완성 개수 컬럼 불필요

6.2 신규 기능: 관리자 동기화 시작

동작 방식

관리자가 차수 페이지에서 "전체 시작" 클릭 → 서버가 단일 트랜잭션으로 처리:

- `rounds.status = active`, `rounds.started_at = T0` (서버 시각)
- 모든 팀에 대해 `attempts(attempt_no = 1, started_at = T0)` 자동 생성

각 팀 모바일은 1초 폴링 중이므로 최대 1초 이내에 상태 변화 감지 → 자동으로 스톱워치 화면으로 전환, T0 기준 카운트업 시작. 팀이 누르는 첫 "정지"는 attempt #1을 종료 처리. attempt #2부터는 팀이 직접 시작/정지로 운영.

왜 정확한가

스톱워치 화면 표시는 폴링 주기 때문에 ± 1 초 정도 차이가 날 수 있지만, 측정 기준은 서버에 박힌 T0이므로 모든 팀의 출발선이 실제로 동일하다. UI만 살짝 지연될 뿐 기록 정확도엔 영향 없음.

오프라인 단말 예외 처리

시작 순간 네트워크 끊긴 단말이 복귀하면 "차수가 [T0]에 시작되었고 남은 시간 [X분]" 안내. 첫 attempt의 `started_at`은 이미 T0로 박혀있어 정합성 유지.

관리자 화면 추가 동작

- 전체 시작 직전에 미접속 팀이 있으면 경고 ("3팀이 아직 미로그인 상태입니다, 진행하시겠습니까?")
- 시작 후에는 차수 종료 시점까지 새 팀 추가는 비활성화 (운영 사고 방지)

6.3 데이터 모델 최종

이전 모델 대비 변경사항:

- `teams.assigned_ft_id` 삭제
- `teams` 테이블에 `UNIQUE(round_id, username)` 추가

- `rounds.started_at` 컬럼 추가 (status=active 전환 시 기록)

6.4 전체 화면 구성

팀(사용자) — 모바일 전용

- TM-01 로그인 (차수 시작 전 미리 로그인 가능)
- TM-02 대기 화면 ("관리자 시작 대기 중", 팀명/차수명)
- TM-03 측정 화면 — 메인. 큰 스톱워치, 15분 카운트다운, 정지 버튼, 직전 시도 요약
- TM-04 시퀀스 입력 — 정지 직후: 조립 순서 5칸 + 턴수 + 메모 + 저장
- TM-05 시도 이력 — 본 팀 attempts 목록, 성공 표시, 읽기 전용
- TM-06 세션 종료 화면 — 15분 후 본 팀 결과 요약

FT — 모바일/태블릿

- FT-01 로그인
- FT-02 차수 선택
- FT-03 팀 목록 — 카드형, 정렬/필터/검색
- FT-04 팀 상세 — 시도 목록, 시퀀스별 "성공 처리" 버튼, FT 메모
- FT-05 그리드 뷰(옵션) — 여러 팀을 한 화면에 동시 표시

관리자 — PC 브라우저 주력

- ADM-01 로그인
- ADM-02 차수 관리
- ADM-03 차수 라이브 운영 — 핵심 화면
- ADM-04 팀 관리 — 차수 내 팀 생성(개별/일괄), 계정 발급
- ADM-05 계정 관리
- ADM-06 기록 조회 / 엑셀 내보내기
- ADM-07 비교 리포트

공개 리더보드 — 큰 화면용

- LB-01 차수 코드 입력 (이후 7절에서 폐기)
- LB-02 리더보드 메인 — 큰 폰트, 시상식 친화 디자인

7. 추가 조정 — TM-03 시작 버튼 및 리더보드 팝업

요청자

팀(사용자)에 스톱워치에 기록 재측정 시 사용할 시작 버튼도 필요(TM-03).

공개 리더보드는 관리자의 차수 별 화면에서 별도 버튼 클릭 시 브라우저 팝업으로 나오면 좋겠음.

다음 단계 선택: (가) ADM-03(라이브 운영) + TM-03(측정 화면) 핵심 화면부터 상세 와이어프레임과 동작 시나리오로 진행.

두 가지 조정사항 반영하고 들어간다.

TM-03 시작 버튼: 측정 화면이 두 상태를 갖도록 설계 변경. 상태 A(측정 중)는 정지 버튼만, 상태 B(시퀀스 사이 대기)는 시작 버튼만. 첫 시도는 관리자 동기화 시작으로 자동 진입, 두 번째 시도부터는 상태 B에서 시작 버튼 클릭으로 진입.

리더보드 팝업: LB-01(코드 입력 화면) 삭제. 대신 ADM-03에 "리더보드 열기" 버튼을 두고 클릭 시

`window.open()` 으로 새 창. URL은 `/leaderboard/{round_id}?token={share_token}` 형태.

`rounds.public_code` 는 사용자 입력용이 아닌 URL 토큰(랜덤 문자열)으로 의미만 변경, 컬럼 자체는 유지.

8. TM-03 측정 화면 + ADM-03 라이브 운영 상세 설계

8.1 TM-03 측정 화면 — 상세 설계

화면 진입은 두 가지 경로:

- TM-02 대기 화면에서 관리자 동기화 시작 신호를 받으면 → 상태 A로 자동 진입 (attempt #1)
- TM-04 시퀀스 입력 저장 후 → 상태 B로 진입

상태 머신

대기 화면(TM-02) —[관리자 전체 시작]→ 상태 A (attempt #1 측정 중)
상태 A —[정지 버튼]→ TM-04 입력 —[저장]→ 상태 B (다음 시도 대기)
상태 B —[시작 버튼]→ 상태 A (attempt #N 측정 중)
상태 A 또는 B —[15분 만료]→ TM-06 세션 종료

15분 만료 시점에 상태 A로 측정 중이면 → 자동 정지 + TM-04 강제 모달 띄움 (마지막 시도 데이터 저장 보장).

와이어프레임 — 두 상태 비교

상태 A — 측정 중 (attempt 진행)

3팀 ● 동기화됨

남은 세션 시간
07:37

시도 #2 측정 중
00:42.31

정지

직전 #1 · 01:23 · 빨노초파흰 · 3T+2

상태 B — 다음 시도 대기

3팀 ↑ 저장됨

남은 세션 시간
06:55

직전 시도 (#2)
00:51.42
빨-초-노-흰-파 · 2T+3 · ✓ 성공

총 시도
2

성공
1

최단
00:51

시작

TM-03 동작 명세

버튼 동작

상태 A의 정지 버튼: 클릭 즉시 클라이언트 시각으로 `stopped_at` 캡처 → IndexedDB에 attempt 레코드 `commit(client_uuid 발급)` → TM-04 입력 페이지로 전환. 정지 클릭부터 입력 페이지 도착까지 어떤 비동기 작업도 끼우지 않음 (정지 시각의 정확성이 가장 중요).

상태 B의 시작 버튼: 클릭 즉시 클라이언트 시각으로 `started_at` 캡처 → IndexedDB에 새 attempt 생성 → 상태 A로 전환.

카운트다운 표시

남은 시간은 `rounds.started_at + time_limit_seconds - now()` 로 클라이언트가 매초 갱신. 음수가 되면 즉시 자동 정지 트리거. 폰을 슬립 후 재시작해도 서버 기준 시각이라 정확.

네트워크 상태 표시 (우측 상단)

세 가지 상태 아이콘:

- `ti-wifi` (초록) — 온라인 + 동기화 큐 비어있음
- `ti-cloud-upload` (회색, 깜빡) — 동기화 중
- `ti-cloud-off` (주황) — 오프라인, 큐에 N건 대기

TM-04로의 데이터 인계

상태 A → TM-04 전환 시 URL 파라미터 또는 라우터 state로 `client_uuid` 전달. TM-04는 IndexedDB에서 해당 attempt를 찾아 추가 데이터(조립 순서 5칸, T값, 추가값, 메모)를 채워 업데이트.

시도 무효화 정책

모든 start-stop 시퀀스는 무조건 저장. TM-04에서 조립 순서/턴수가 비어있어도 저장 가능. 단 사용자에게 "조립 순서를 입력하지 않으면 FT가 성공 처리하기 어렵습니다" 안내 표시.

8.2 ADM-03 차수 라이브 운영 — 상세 설계

관리자가 시작-종료까지 한 화면에서 끝낼 수 있도록 설계. 폴링 주기는 1.5초(팀 카드 갱신), 0.5초(카운트다운).

와이어프레임

2026 상반기 신입교육 1차

남은 시간

진행 중

시작 14:32:18 · 6팀

07:42

▶ 전체 시작

↗ 리더보드 열기

■ 차수 종료

↓ 엑셀 내보내기

총 팀

6

측정 중

4

시도 합계

18

성공

3정렬:

최단기록

시도수

성공수

팀명

1팀

측정 중

시도 5

성공 1

01:14

3팀

측정 중

시도 2

성공 1

01:23

5팀

측정 중

시도 4

성공 1

01:38

2팀

시퀀스 대기

시도 3

성공 0

-

4팀

측정 중

시도 3

성공 0

-

6팀

미접속

시도 1

성공 0

-

ADM-03 동작 명세

액션 버튼 활성화 조건

버튼	preparing	active	ended
전체 시작	활성	비활성	비활성
리더보드 열기	비활성	활성	활성
차수 종료	비활성	활성	비활성
엑셀 내보내기	비활성	활성(중간 확인용)	활성

전체 시작 클릭 시 흐름

- 확인 모달 → "6팀 중 1팀이 미접속 상태입니다. 진행하시겠습니까?"
- 확인 → 서버 POST `/api/rounds/:id/start`

3. 단일 트랜잭션으로 `rounds.status = active`, `rounds.started_at = T0`, 각 팀에 `attempts(attempt_no=1, started_at=T0)` 일괄 INSERT
4. 응답 → 화면이 active 모드로 갱신

서버에서 T0를 박는 이유: 관리자 브라우저와 팀 단말 간 시계 차이 제거. 모든 측정의 기준은 서버 T0.

리더보드 열기 클릭

```
window.open(`/leaderboard/${roundId}?token=${shareToken}`,
  '_blank', 'width=1280,height=800')
```

별도 브라우저 창으로 열림. 시상식 디스플레이용이라 큰 모니터에서 전체화면(F11) 사용 권장 — 안내 톨팁 표시.

차수 종료 (수동/자동)

수동: 관리자가 종료 버튼 클릭 → 확인 모달 → `status=ended`, `ended_at=now()` 기록 → 모든 팀의 진행 중인 attempt가 있으면 서버에서 `stopped_at`을 `ended_at`로 보정 + 자동 정지 신호 전파.

자동: 서버는 `now() >= started_at + time_limit_seconds` 인 차수가 active이면 어떤 API 호출에서든 종료 처리(lazy 종료). 클라이언트도 카운트다운 0 도달 시 종료 API 호출. 이 lazy 종료 방식은 Cloudflare Workers에 별도 cron 없이 동작 가능 — 가장 가벼움.

팀 카드 상태 색상

상태	색상	의미
측정 중	초록	attempt가 active (stopped_at 없음)
시퀀스 대기	주황	직전 attempt 정지됐고 다음 시작 안 함
미접속	회색	30초 이상 폴링 미수신 (heartbeat 기반)
종료	진회색	차수 종료 후

8.3 차수 라이프사이클

```
[preparing] —[관리자 전체 시작]→ [active] —[수동/자동 종료]→ [ended]
    |                               |
    └─ 차수/팀 생성, 시간 설정,   └─ 시도 누적, FT 성공 처리,
      계정 발급 자유              실시간 모니터링, 리더보드 송출
```

차수가 `ended` 로 들어가면 모든 쓰기 차단 (시도 저장, 성공 처리, FT 메모 모두 거부). 읽기와 엑셀 내보내기만 허용.

8.4 핵심 시나리오 5개

시나리오 1 — 동기화 시작

관리자가 "전체 시작" 누름 → 서버 T0 기록 → 각 팀 단말 1초 이내 폴링으로 감지 → TM-02에서 TM-03 상태 A로 자동 전환 → 모든 팀 스톱워치가 동일한 T0 기준으로 카운트업 시작.

시나리오 2 — 시도 1회 사이클

팀이 정지 누름 → 클라이언트 stopped_at 캡처 → IndexedDB commit → TM-04 입력 페이지 → 조립 순서 + 턴 수 입력 + 저장 → 서버 동기화 → 상태 B로 진입 → 시작 누름 → 새 attempt 시작.

시나리오 3 — FT 성공 처리

FT가 FT-03 팀 목록에서 3팀 클릭 → FT-04 시도 목록에서 #2를 "성공 처리" → 서버 `is_success=true` 갱신 → ADM-03 다음 폴링(1.5초 이내)에 3팀 카드 "성공 1 → 2", 최단기록 갱신 → 리더보드 다음 폴링(1초)에 순위 재계산.

시나리오 4 — 네트워크 단절 복구

팀 단말 와이파이 끊김 → 정지 버튼 동작 정상 → IndexedDB에 attempt + client_uuid 적재 → 화면 우상단에 "오프라인, 큐 3건" 표시 → 시작 버튼도 정상 동작 → 와이파이 복구 → 백그라운드 동기화로 큐 순차 전송 → 서버는 client_uuid 중복 체크하여 먹등 처리.

시나리오 5 — 15분 만료

서버 시각이 T0 + 900초 도달 → 클라이언트 카운트다운이 먼저 0에 도달 → 자동 정지 + TM-04 강제 모달 → 관리자 ADM-03도 자동으로 ended 상태로 갱신 → 더 이상 새 attempt 저장 안 됨 → 리더보드 최종 결과 고정.

9. Claude Code CLI 연동 산출물

요청자

이걸 claude code CLI에서 진행하려고하는데 그 쪽으로 모든 정보를 연결할 수 있을까?

가능. Claude Code CLI는 프로젝트 루트의 `CLAUDE.md` 를 세션 시작 시 자동으로 읽고, `docs/` 하위 문서들도 필요할 때 참조한다. 두 개의 마크다운 파일로 패키징했다.

산출물

CLAUDE.md (프로젝트 루트) — Claude Code가 매번 자동으로 읽는 컨텍스트. 시스템 개요, 기술 스택, 핵심 설계 원칙, 폴더 구조 제안, 작업 컨벤션, 첫 작업 순서.

docs/SPEC.md — 상세 명세서. 데이터 모델(SQL DDL), API 엔드포인트, 모든 화면 명세, ADM-03/TM-03 상세, 핵심 시나리오 5종, 엣지 케이스, 개발 로드맵.

두 파일은 별도 다운로드 가능 (이 PDF와 함께 제공).

9.1 권장 폴더 구조

```
your-project/
├── CLAUDE.md      ← 자동 다운로드 파일
└── docs/
    └── SPEC.md    ← 자동 다운로드 파일
```

9.2 Claude Code 시작 방법

- 프로젝트 폴더 만들고 위 두 파일 배치
- 터미널에서 `cd your-project && claude` 실행
- Claude Code가 시작되면서 `CLAUDE.md` 를 자동으로 컨텍스트에 로딩
- 첫 메시지로 다음과 같이 시작:

CLAUDE.md와 docs/SPEC.md를 읽고 시스템 개요를 한 문단으로 요약해줘.
이상하거나 모호한 부분이 있으면 지적해주고.
이후 Phase 1 작업(인프라 셋업 + 레포 구조 + 인증 골격)을 시작할 거야.

9.3 작업 진행 팁

- Phase 단위로 끊어서 진행** — SPEC.md 10절 로드맵의 Phase 1부터 차례대로. 각 Phase 끝에 검증 항목이 있으니 동작 확인 후 다음으로
- 새 화면 작업 시작 시:** "TM-03 특정 화면을 구현할거야. docs/SPEC.md의 6절 상세 명세를 따라줘" 같이 화면 ID로 가리키면 됨

- **결정이 필요한 시점:** CLAUDE.md 끝의 "알려진 결정 미정 사항"을 참고해서 그때그때 결정. 결정 사항이 누적되면 두 문서에 반영
- **데이터 모델 변경 시:** SPEC.md 2.2절 SQL부터 수정 → 마이그레이션 파일 생성 → 코드 반영 순서로
- **Claude Code 세션 간 컨텍스트 유지:** CLAUDE.md 자체에 결정사항/진행상황을 누적 기록하면 다음 세션에서도 일관성 유지됨. Claude Code에게 "오늘 결정한 사항을 CLAUDE.md에 추가해줘"라고 요청 가능

9.4 SPEC.md 주요 구성

1. 시스템 아키텍처
2. 데이터 모델 (CREATE TABLE 문 포함)
3. API 엔드포인트 (인증, 차수, 팀, 시도, FT 메모, 리더보드)
4. 화면별 기능 명세 (TM/FT/ADM/LB 전 화면)
5. ADM-03 라이브 운영 상세
6. TM-03 측정 화면 상세 (코드 스니펫 포함)
7. 핵심 시나리오 5종
8. 엣지 케이스 (동시성, 시간, 데이터 정합성, UI)
9. 보안 및 운영 (PBKDF2, JWT, share_token, Rate Limit, CORS, 로깅, 보존)
10. 개발 로드맵 (Phase 1~8, 약 16-21일 예상)

개발 일정 추산

단일 풀스택 개발자 기준 약 16-21일, 프론트/백 분담 시 약 12-15일로 단축 가능. 가장 까다로운 구간은 Phase 3 (ADM-03 동기화 시작 트랜잭션 + TM-03 측정 화면).